



Slide 1
CS 170
Java Programming 1
The JavaDocs
 Duration: 00:00:59
 Advance mode: Auto

Notes:

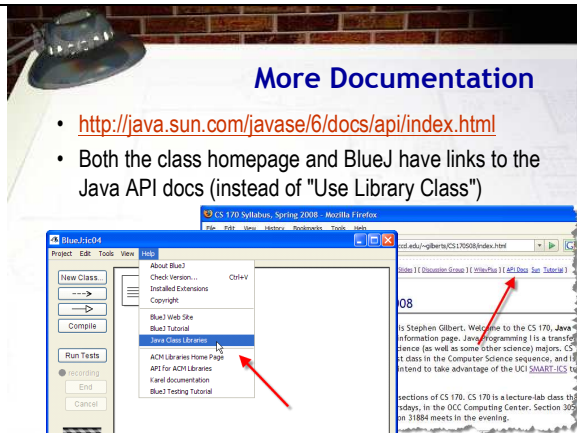
Hi Folks. This is the CS 170, Java Programming 1 lecture, The JavaDocs

There's an old saying, you've probably heard a hundred times:

Give a man a fish and you feed him for a day. Teach a man to fish, and you feed him for his life.

Programming is a lot like that. Learning about Rectangle and String and Color objects is not nearly as important as learning how to learn about Rectangle and String and Color objects. In Java, you can learn about all the Java classes by using the online Java Documentation, often called the JavaDocs after the program which generates the HTML help files from comments embedded in the Java source code.

Of course, you've already met the JavaDocs for the Rectangle class in the last few lessons. In this lesson, though, you'll learn how to get to the docs from the Web, how to search for the class you want to find, and how to interpret the documentation that you find there.



Slide 2
More Documentation
 Duration: 00:01:19
 Advance mode: Auto

Notes:

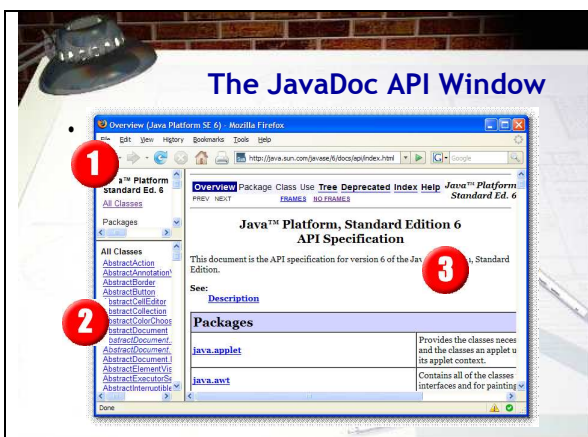
You can get started by using your browser to access the online documentation at Sun. The online docs are always available at:

<http://java.sun.com/javase/6/docs/api/index.html>

Of course, sometimes it's hard to remember something like that, so both the class home page and BlueJ have links to the Java API docs as well.

On the class syllabus page you'll find the link in the upper-right corner; it should look something like this. In BlueJ, there's a link on the Help menu that says Java Class Libraries.

If you aren't connected to the Internet, or, if you have a slow connection, you might want to read the documentation on your local machine. Simply double-click the file index.html, located in the docs/api directory of your JDK installation. The JDK docs directory is not automatically installed when you install the JDK. You have to download and install the docs separately. See Sun JDK installation site for more information.



Slide 3

The JavaDoc API Window

Duration: 00:01:14

Advance mode: Auto

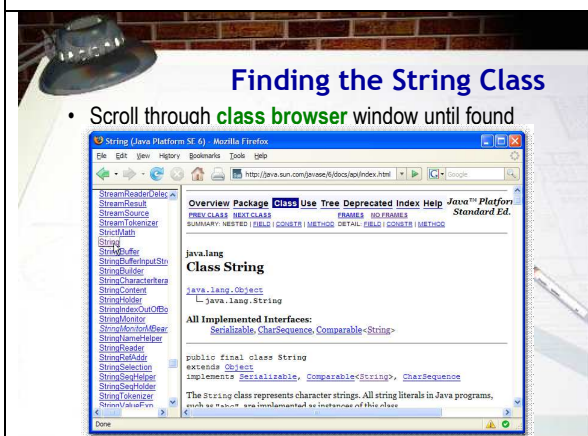
Notes:

Once you open the API documentation, you'll start at the main JavaDoc window, which you can see below. The JavaDoc interface consists of three frames:

First, the package browser in the upper left-hand corner allows you to select individual packages, so that only the classes in that package appear inside the class browser window below. You can also click individual packages by selecting the links in the center frame. I usually close this frame by sliding the bottom bar up to the top, since I don't find it very useful to restrict my searching to a single package. When you first arrive, the package browser is set to display all classes.

Second, the class browser in the lower-left corner consists of hyperlinks to all of the classes and interfaces in the currently selected package. Since these are in alphabetical order, it's a simple matter to scroll down and find the class you want.

Finally, the content pane in the center is where you'll find the documentation for the class you're interested in. Let's look at the documentation for a class we've already met, the String class.



Slide 4

Finding the String Class

Duration: 00:00:45

Advance mode: Auto

Notes:

To find the String class, simply scroll through the class browser frame until you find the name. If you click in the frame you can also use your Browser's search or find function.

When you find the String class, click the hyperlink, and the documentation for the class will be displayed in the main content pane, as you can see here.

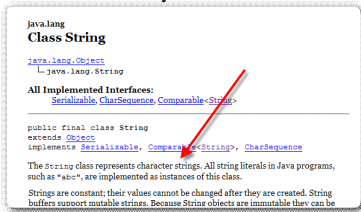
Finally, some meaningful information! The JavaDoc for each class is broken up into several sections. Most of these won't make a lot of sense to you until you've learned a little more about Java. For this lecture we're only concerned about three sections:

1. Introduction and Class Summary
2. Method Summary
3. Method Details

Let's start with the introduction.

Class Summary

- Locate the inheritance tree and the package name
 - Use the package in your **import** statement
 - The **java.lang** package is **automatically** imported
- Read the class summary for an overview of the class



Slide 5 

Class Summary

Duration: 00:01:21

Advance mode: Auto

Notes:

At the very top of every class documentation page, there is an introduction to help you get oriented. The introduction starts with the class name (Class String in the picture) and is followed by a visual "ladder" of inheritance. This is important because some methods may be inherited from a superclass, so this gives you a place to start looking.

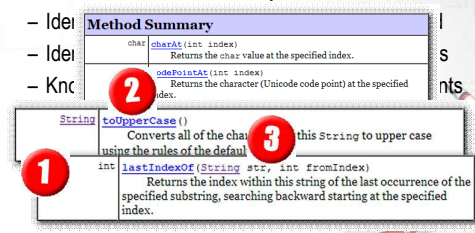
Right above the class name is the name of the package that contains the class; in the case of the String class, that's the java.lang package. The java.lang package is automatically imported into every Java program, but for most classes, you'll use the package name you find here when you write your import statements.

Right below the inheritance hierarchy, you'll normally find more introductory information. Here's the very beginning of the introductory section for the String class. For most classes you'll find:

- A simple explanation that tells you what the class does and, often more important, why it exists.
- A simple code example. For some of the more complex classes, the explanation is not enough; a code example is "worth its weight in gold", (to worry an old cliché).
- A picture of the component for the basic visual classes.

Deciphering the Docs

- Read method documentation to "talk" to your objects
 - Locate the method summary
 - Identify the return type
 - Identify the method name
 - Know the parameters



Slide 6 

Deciphering the Docs

Duration: 00:01:57

Advance mode: Auto

Notes:

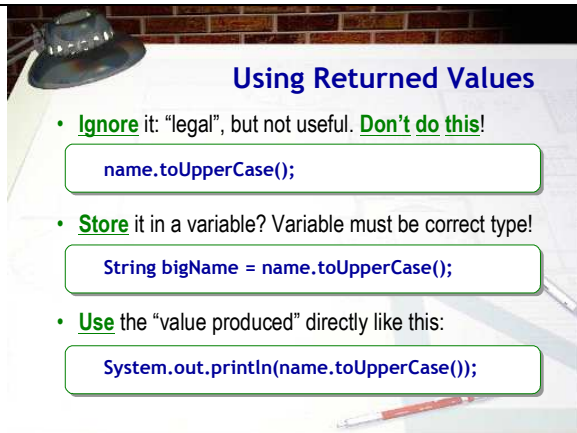
In the last few lessons you've used different kinds of methods to "talk" to the objects you created. The documentation provides the tools you'll need to make yourself understood. The "dictionary" of commands that any given object understands is listed in its method summary. These are the things that you can tell a String object to do or to calculate. Let's take a closer look at two different String examples: the toUpperCase() method and the lastIndexOf() method.

First, you should be able to identify the return type column (which we used to differentiate between mutator methods and accessor methods), and the method name (which you'll use to call the method).

Next, you should be able to identify elements of the parameter list. As you can see in the picture, there are no parameters defined for the toUpperCase() method, so you don't need to supply any additional information when you call the method. The lastIndexOf() method, though, has two parameters defined.

The first parameter is a String, and the second parameter is an int. That means that when you call the lastIndexOf() method, inside of the parentheses you have to supply a String value (either a variable or text between double quotes), followed by an integer value, either a variable or a literal whole

	number like 17. You'll separate the two values (which are called actual parameters, recall), with a comma. Just to recap: • If you see a parameter with a type of String, put "text between double quotes" in that position • If you see a parameter with a type of int, put a whole number in that position • If you see a parameter with a type of double, put a real number (that is, a number with a decimal point) in that position
Express Yourself • Exercise 1: What import statement would you have to add to your program if you wanted to use the TimeZone class? How or where would you find the correct import? Shoot a screenshot showing me. Slide 7 🎧 Express Yourself Duration: 00:00:43 Advance mode: Auto	Notes: OK, now it's your turn. Start a new section in your in-class lab exercise document. The first few questions in this lesson will ask you to write some code, but you don't need BlueJ to do it; you can type your code directly into the lab exercise document. We'll use BlueJ for the later questions. Using only what you've learned in the last few lessons, for Exercise 1, tell me what import statement you'd need to add to your programming if you wanted to use the TimeZone class. How or where would you find the correct import? Shoot a screen-shot showing me.
Express Yourself • Exercise 2: What mutator methods could you call, assuming you only knew how to pass String, int and double parameters? • Exercise 3: Your program contains a TimeZone object named tz, write statements to call each of the methods from the last question. The actual values you use are unimportant if you use the right "kind" of thing. Slide 8 🎧 Express Yourself Duration: 00:01:08 Advance mode: Auto	Notes: Now that you've found the TimeZone docs, for Exercise 2, look through them and tell me which mutator methods you could call, assuming that you only know how to pass String, int, and double parameters? For Exercise 3, let's assume that your program already contains a TimeZone variable named tz, and that you've created and "hooked up" a TimeZone object as well. In other words, tz is ready to go. For every mutator method you identified in Exercise 2, use the variable tz and write a statement that calls the method. You don't actually have to know what the methods mean in this case; the values you pass aren't important as long as they are the right "kind of thing". Each of these methods is a mutator, so you call them by simply using the name of the method, a set of parentheses containing the parameter values and a semicolon. You can't use the method in any other context. That's not true with accessors and other kinds of functions, though.



Using Returned Values

- **Ignore** it: "legal", but not useful. **Don't do this!**

```
name.toUpperCase();
```
- **Store** it in a variable? Variable must be correct type!

```
String bigName = name.toUpperCase();
```
- **Use** the "value produced" directly like this:

```
System.out.println(name.toUpperCase());
```

Slide 9

Using Returned Values

Duration: 00:01:30

Advance mode: Auto

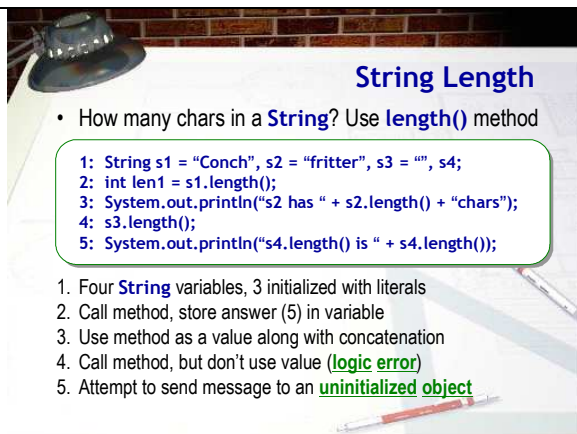
Notes:

Remember that accessors (and other functions) give you back (or return) some information when you invoke the method. When you call an accessor, there are actually three syntactically correct ways to call them.

First, you can call the accessor method just like you would call a mutator method; that is, you can just ignore the information that the method returns. While this doesn't create a syntax error (that is, the compiler thinks that it's just fine), it is a logical error, since your code has no effect. In this case, for instance, you might think that name would be changed to upper case, but since toUpperCase() is an accessor, not a mutator, name isn't changed at all. The method does return an upper-case copy of the variable name, but since we don't do anything with it, it's just discarded.

You can save the information returned by a method by storing it in a variable. The only caveat is that you have to remember to create the right kind of variable. Since the toUpperCase() return type is String, then the variable bigName must also be a String.

Finally, you can simply use the value directly, just as if it were a value. You already saw how this worked in Code Pad. Here's an example that passes the upper-case copy of name to System.out.println() to display it.



String Length

- How many chars in a **String**? Use **length()** method

```
1: String s1 = "Conch", s2 = "fritter", s3 = "", s4;  
2: int len1 = s1.length();  
3: System.out.println("s2 has " + s2.length() + "chars");  
4: s3.length();  
5: System.out.println("s4.length() is " + s4.length());
```

1. Four **String** variables, 3 initialized with literals
2. Call method, store answer (5) in variable
3. Use method as a value along with concatenation
4. Call method, but don't use value (**logic error**)
5. Attempt to send message to an **uninitialized object**

Slide 10

String Length

Duration: 00:01:31

Advance mode: Auto

Notes:

Let's look at the different ways you can use an accessor method by using the String length() method as an example. The length method returns the number of characters in a String, which may be 0.

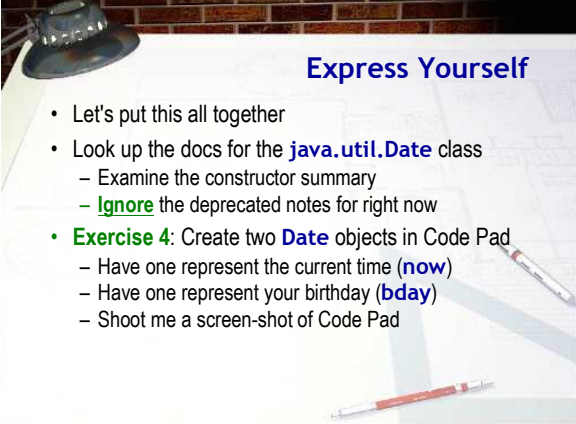
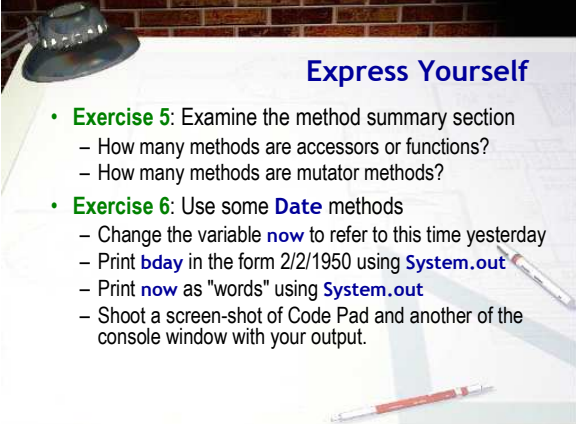
Line 1 creates four String variables, three of them initialized with literals. Notice that s4, though, is a String variable but it doesn't refer to a String object. See if you can figure out what happens with the remaining lines.

Line 2 calls the method and stores the value it gets back in an int variable, since the length() method returns an int.

Line 3 doesn't store the value it gets back, but uses it directly, along with String concatenation to print out the result.

Line 4 calls the length() method as if it were a mutator; it simply ignores the returned value, so nothing happens. Even though this isn't a syntax error, it is a logic error.

Line 5 is also syntactically correct and also is an error. A logic error, like that in line 4, occurs when your program produces the wrong output. A runtime error, that that generated by this line, occurs when your program "crashes" and prints out an error message. In this case, the error is caused by trying to send a message to a variable that hasn't been "hooked up" to an object.

 Express Yourself • Let's put this all together • Look up the docs for the <code>java.util.Date</code> class – Examine the constructor summary – Ignore the deprecated notes for right now • Exercise 4: Create two <code>Date</code> objects in Code Pad – Have one represent the current time (<code>now</code>) – Have one represent your birthday (<code>bday</code>) – Shoot me a screen-shot of Code Pad Slide 11 🎧 Express Yourself Duration: 00:01:40 Advance mode: Auto	Notes: Now, let's put the last four lessons all together. Break out BlueJ and let's see if you can create some object variables, create and assign some objects, call some mutators and get some values from some accessors. Of course, I'll want you to do that with a class you haven't yet met. You'll get all of the information you need by exploring the Java docs. The class I want you to look up is the <code>java.util.Date</code> class. There's another <code>Date</code> class in the <code>java.sql</code> package; check carefully to make sure you have <code>java.util.Date</code>. We'll start by creating object variables and objects, so locate the constructor summary section. (Also, some of the methods we're going to use in this class are deprecated, meaning that Java has better classes or methods for you to use. We're going to ignore that for right now.) For Exercise 4 create two <code>Date</code> variables and initialize them with objects using the most appropriate constructor. You'll use a different constructor for each variable. The first variable should be named <code>now</code> and you should initialize its <code>Date</code> object to contain the current date and time. The second variable should be named <code>bday</code> and should be initialized to your birthday. Read the detailed constructor documentation to make sure the values you supply as parameters mean what you think that they mean.
 Express Yourself • Exercise 5: Examine the method summary section – How many methods are accessors or functions? – How many methods are mutator methods? • Exercise 6: Use some <code>Date</code> methods – Change the variable <code>now</code> to refer to this time yesterday – Print <code>bday</code> in the form 2/2/1950 using <code>System.out</code> – Print <code>now</code> as "words" using <code>System.out</code> – Shoot a screen-shot of Code Pad and another of the console window with your output. Slide 12 🎧 Express Yourself Duration: 00:01:18 Advance mode: Auto	Notes: For Exercise 5, look through the method summary for the <code>Date</code> class. How many of the methods are accessors or functions? How many of the methods are mutator or effector methods? Finally, to finish up for Exercise 6, let's write some code in Code Pad that uses your two <code>Date</code> variables. These will take some thought and a careful study of the available methods. • Change the variable <code>now</code> to refer to this time yesterday. You'll need to use both an accessor and a mutator. • Print <code>bday</code> on the screen in the form 2/2/1950 using <code>System.out</code>. There is no built-in method to do this. You'll have to write several statements using the <code>print</code> method along with different accessors. • Print the variable <code>now</code> as "words" using <code>System.out</code>. There

	are several methods in the Date class that will do this for you. When you're finished with these, shoot a screen-shot of the Code Pad window and another one of the console window with the output from using System.out.
--	---